KB Update Principle

Date: 2026-07-03

Status: complete, pending owner acceptance

Executive Rule

The knowledge base exists so that a future agent can act correctly without reading chat history. Every KB write must therefore answer one question: "Will a future agent make a better decision because this exists?" If yes, write it to the correct layer with provenance. If no, leave it as transient run evidence or do not record it at all.

WikiLLM is the only durable memory. LlamaIndex is retrieval only. Graphify is generated structural reference only. Dashboard packets, Notion rows, Obsidian/Nexus notes, and Telegram deliveries are surfaces, not memory: nothing on a surface counts as knowledge until an agent reviews it and writes it into WikiLLM.

All KB content is public-safe English. No secrets, private URLs, account IDs, local absolute paths, raw transcripts, screenshots, or personal identifiers ever enter any KB layer.

Layer Responsibilities

Layer	Role	Write authority	Durability
WikiLLM (`wiki/`)	Canonical durable public-safe project memory	Agents, after review, through the writing rules below	Durable
Run artifacts (`project/runs/<run-id>/`)	Transient evidence: handouts, packets, drafts, check output	The executing agent	Durable as evidence, not as memory
Reports (`project/reports/`)	Reviewed method and analysis documents	The executing agent, after validation	Durable reference
LlamaIndex	Bounded hybrid retrieval over the approved corpus (`project/`, `history/`, `skills/`, `wiki/`)	None; index is rebuildable, never authoritative	Disposable
Graphify (`graphify-out/`, `project/reference/graphify/`)	Generated structural navigation reference	Generation scripts only	Disposable, regenerable
Obsidian/Nexus	Human semantic layer; live bridge gated until schema/tool proof exists in the current run	None by default; public-safe summaries only after discovery proof	Mirror only
Notion	Task/PM board and reporting surface	Append evidence; never flip status without owner approval	External surface
Dashboard packets (`project/dashboard/`)	Operator command center and review-packet display	Generated by `generate-dashboard-data.py`; browser-local state is never memory	Regenerated
Telegram	External delivery path via approved sender scripts	Sanitized delivery status only	Delivery proof only

What Becomes Durable Knowledge

Promote to WikiLLM only content that changes future agent behavior:

- Corrected assumptions and reusable constraints (memory).
- Reusable analytical meaning: why an approach works, sequencing logic, boundary insights (insights).
- Completed substantial runs: what was done, checked, and left open (runs).
- Unresolved risks or broken states another agent must know about (issues).
- Accepted or superseded architecture and method choices with context (decisions).
- Run completion pointers with dates (log).

The test for promotion: the fact must be stable across runs, stated in public-safe English, and traceable to a run artifact or check output.

What Does Not Become Durable Knowledge

- Raw source material of any kind, private or public. Summaries only.
- Chat dialogue, intermediate drafts, tool call transcripts, or model output not grounded in reviewed sources.
- Dashboard packets, voice captures, or browser-local state before an agent has reviewed them.
- Anything already derivable from the repository: file listings, code structure, git history.
- Duplicate statements of existing memory; update the existing entry instead.
- Speculation without a FACT base; if recorded at all, it must be labeled HYPOTHESIS inside a run note, not memory.
- Secrets, credentials, tokens, private URLs, IDs, local absolute paths, personal identifiers, raw transcripts, screenshots.
- Claims about customers, revenue, demand, ROI, or production outcomes without public evidence.

Source And Provenance Rules

- Every durable statement must carry a repo-relative source reference (file path, run id, or check name). "Where does this come from?" must always be answerable.
- Private sources may be read for understanding, but only translated public-safe English summaries enter tracked files, with the source described generically (for example "owner planning discussion, 2026-07").
- Approved corpus for retrieval is exactly `project/`, `history/`, `skills/`, `wiki/`. Widening it silently is forbidden.
- Provider or model output is never a source of fact. It becomes knowledge only after source-grounded agent review.
- Dates are absolute (YYYY-MM-DD), never relative.

Retrieval Rules

- LlamaIndex runs in hybrid bounded mode with lexical fallback; vector defaulting stays gated until embedder/vector readiness is proven.
- Retrieval outputs must include source paths. A retrieval result without a source path is not usable evidence.
- Retrieval is read-only. Nothing retrieved is rewritten into memory unless it passes the promotion test above.
- Refuse retrieval requests over private sources or paths outside the approved corpus.

- Checks: `project/scripts/llamaindex-approved-corpus.py --mode smoke` and `project/scripts/llamaindex-rag-benchmark.py` must pass after retrieval-affecting changes.

WikiLLM Writing Rules

Decide the target file by the type of durable meaning:

Target	Write when	Do not write when
`wiki/memory.md`	A reusable future constraint changed, or an assumption was corrected	The fact is run-specific or already recorded
`wiki/insights.md`	A reusable analytical conclusion emerged that explains why, not just what	It restates memory or describes one-off mechanics
`wiki/runs/<run-id>.md`	Any substantial run completed (new artifacts, method change, multi-file work)	Trivial single-file mechanical edits
`wiki/issues/`	A risk or broken state persists past the run and needs an owner or agent decision	The problem was fixed inside the run (record in the run note)
`wiki/decisions/`	An architecture or method choice was accepted, proposed, or superseded	The choice is reversible run-local tactics
`wiki/log.md`	Every run note gets a one-line dated pointer	Never write content here, only pointers

Keep entries short. A memory entry is one to three sentences. An insight is one paragraph. Length lives in run artifacts and reports, not in memory.

Dashboard Packet Rules

- The dashboard displays state; it does not create knowledge. `project/dashboard/data.json` is regenerated from tracked files by `generate-dashboard-data.py`, never edited by hand.
- Review packets exported from the dashboard (PRD/ICP mode or agent-orchestra mode) are candidate evidence. They become durable knowledge only after an agent reviews them, verifies provenance, and writes a summary into the correct WikiLLM file.
- Dashboard writeback to GitHub, Notion, WikiLLM, Obsidian, or Telegram remains disabled. Do not claim otherwise.
- If public text changed in a run, regenerate dashboard data and re-parse the JSON before closeout.

Notion And External Sync Rules

- Notion is a reporting surface. Agents append evidence and prepare update candidates; owner approval is required to flip task status to Done.
- Keep task numbering, Agent Tags, and GitHub links coherent; never store private Notion page URLs in public files.
- External sync claims require executed connector proof in the run note (tool listed, schema inspected, read before write, sanitized result recorded). No executed call means the run note records "external sync skipped."
- Telegram deliveries go only through the approved sender scripts with credentials in ignored local env; only sanitized delivery status (file name, timestamp, ok/fail) is recorded.

Error And Traceback Rules

When a wrong action, bad capture, credential-boundary mistake, or provider/runtime confusion occurs:

- Stop the affected lane. Do not build further writes on top of the error.
- Record the event in the current run note with FACT (what happened), INTERPRETATION (impact), GAP (what remains unknown).
- Correct the durable layer: fix or remove wrong statements in `wiki/memory.md` or `wiki/insights.md` directly, and note the correction in the run note. Never leave known-wrong memory standing.
- For credential-boundary mistakes (secret, private URL, ID, or local path in a tracked file): remove the content, run `scripts/public_safety_scan.py`, record the incident in `wiki/issues/`, and flag to the owner before any push. If a secret value was exposed, recommend rotation; the KB stores only the masked incident record.
- For bad captures (wrong source, unreviewed packet promoted, misattributed claim): demote the content back to run evidence, restate with correct provenance, and add an insight if the failure mode is reusable.
- For provider/runtime confusion (claiming a runtime, provider, or writeback works without proof): correct the claim everywhere it appears, and re-verify against the gated-claims list (provider-backed Jarvis, Railway, live Nexus, vector defaulting, dashboard writeback, raw voice storage, autonomous external updates).
- Traceback must be possible from the wrong statement to its origin run via the log pointer and run note. This is why every durable statement carries provenance.

Agent Workflow

For any run that may touch the KB:

- Preflight: `git status`, read operating rules, live communication files, and the latest run notes; append a `starting` entry with file claims.
- Execute the bounded task; keep all drafts and evidence under `project/runs/<run-id>/`.
- Label reasoning that changes state or risk as FACT / INTERPRETATION / HYPOTHESIS / GAP. Facts need provenance; hypotheses never enter memory.
- At closeout, apply the promotion test to everything produced: run note always (if substantial), memory/insight/issue/decision only if the rules above say so, log pointer always with the run note.
- Update the run handout (`agent-handout.md`) per the task-handout skill.
- Regenerate dashboard data if public text changed.
- Run the validation bundle for the change class (documentation, workflow/RAG, dashboard, or API).
- Append a `complete`, `blocked`, or `handoff` entry to the live communication log with files changed, checks run, gaps, and next safe action.
- Commit only when required; push only when approved and checks pass.

Validation

For KB and documentation changes, the minimum bundle is:

- `python3 scripts/public_safety_scan.py`
- `python3 project/scripts/generate-dashboard-data.py` plus JSON parse of `project/dashboard/data.json`

- `git diff --check`

If workflows, runtime, skills, or RAG were touched, add: `validate-workflows.py`, `pre-push-runtime-guard.py`, `llamaindex-approved-corpus.py --mode smoke`, `llamaindex-rag-benchmark.py` (all via `project/local/venv/bin/python`). If the venv is unavailable in the current environment, record the skipped checks as an explicit GAP and do not claim them.

Acceptance Criteria

This principle is acceptable when:

- Public-safe boundaries are preserved in every rule above.
- WikiLLM is defined as the only durable memory and LlamaIndex as retrieval only.
- Graphify stays generated structural reference; Obsidian/Nexus stays gated behind schema/tool proof.
- The decision table specifies exactly when to write memory, insight, issue, decision, run, and log entries.
- Dashboard packets become durable knowledge only after agent review.
- The traceback section covers credential-boundary and wrong-capture corrections.
- A new agent can update the KB correctly from this document alone, without chat history.
- The documentation validation bundle passes.

Next Actions

- Owner review and acceptance of this principle (E1.4 gate).
- Apply this principle during E2.0A: the PRD-to-ICP dry-run packet stays run evidence until reviewed, then promotes per these rules.
- Reference this report from `wiki/index.md` read order after owner acceptance.
- Fold acceptance-confirmed constraints into `wiki/memory.md` as one short entry pointing here.